

Graph Library: Algorithms

Document #: **P3128r3**
Date: 2025-04-13
Project: Programming Language C++
Audience: Library Evolution
SG19 Machine Learning
SG14 Game, Embedded, Low Latency
Revises: P3128r2

Reply-to: Phil Ratzloff (SAS Institute)
phil.ratzloff@sas.com
Andrew Lumsdaine
lumsdaine@gmail.com

Contributors: Kevin Deweese
Muhammad Osama (AMD, Inc)
Jesun Firoz
Michael Wong (Intel)
Jens Maurer
Richard Dosselmann (University of Regina)
Matthew Galati (Amazon)
Guy Davidson (Creative Assembly)
Oliver Rosten

1 Getting Started

This paper is one of several interrelated papers for a proposed Graph Library for the Standard C++ Library. The Table 1 describes all the related papers.

Paper	Status	Description
P1709	Inactive	Original proposal, now separated into the following papers.
P3126	Active	Overview , describes the big picture of what we are proposing.
P3127	Active	Background and Terminology provides the motivation, theoretical background, and terminology used across the other documents.
P3128	Active	Algorithms covers the initial algorithms as well as the ones we'd like to see in the future.
P3129	Active	Views has helpful views for traversing a graph.
P3130	Active	Graph Container Interface is the core interface used for uniformly accessing graph data structures by views and algorithms. It is also designed to easily adapt to existing graph data structures.
P3131	Active	Graph Containers describes a proposed high-performance <code>compressed_graph</code> container. It also discusses how to use containers in the standard library to define a graph, and how to adapt existing graph data structures.
P3337	In process	Comparison to other graph libraries on performance and usage syntax. Not published yet.

Table 1: Graph Library Papers

Reading them in order will give the best overall picture. If you're limited on time, you can use the following guide to focus on the papers that are most relevant to your needs.

Reading Guide

- If you're **new to the Graph Library**, we recommend starting with the *Overview* (P3126) paper to understand the focus and scope of our proposals. You'll also want to check out how it stacks up against other graph libraries in performance and usage syntax in the *Comparison* (P3337) paper.
- If you want to **understand the terminology and theoretical background** that underpins what we're doing, you should read the *Background and Terminology* (P3127) paper.
- If you want to **use the algorithms**, you should read the *Algorithms* (P3128) and *Graph Containers* (P3131) papers. You may also find the *Views* (P3129) and *Graph Container Interface* (P3130) papers helpful.
- If you want to **write new algorithms**, you should read the *Views* (P3129), *Graph Container Interface* (P3130), and *Graph Containers* (P3131) papers. You'll also want to review existing implementations in the reference library for examples of how to write the algorithms.
- If you want to **use your own graph data structures**, you should read the *Graph Container Interface* (P3130) and *Graph Containers* (P3131) papers.

2 Revision History

P3128r0

- Split from P1709r5. Added *Getting Started* section.
- Added A*, Best-first search and Adamic-Adar Index to Tier 2 algorithms based on input.
- Removed allocator parameters for consistency with existing algorithms. It was observed that `stable_sort` allocates memory, but does not take an allocator parameter.
- Removed exception throwing from algorithms to support freestanding C++. The caller will need to follow the preconditions to avoid undefined behavior. The other option considered was to return an error code.

- Identify all **concept** definitions as "For exposition only" until we have consensus of whether they belong in the standard or not.

P3128r1

- Create new *Traversal* section and move Breadth-First Search and Topological Sort algorithms to it. Also added new Depth-First Search algorithm to it.
- Revise the Dijkstra and Bellman-Ford shortest-path and shortest-distance algorithms
 - Add a visitor parameter to allow the caller to customize the behavior of the algorithm without having to modify the algorithm itself. The visitor functions are member functions on a user-defined class that must match the concept for the event. The visitor events mimic those used in the Boost Graph Library for each algorithm.
 - Remove overloads that excluded Compare and Combine functions because they don't add much value, and to keep the proposal small.
 - Add overloads for multiple sources. This is particularly important for Bellman-Ford to avoid repeated calls to the algorithm that would make an already slow algorithm even slower.
 - Change "invalid distance" to "infinite distance" to reflect how the value is used in the algorithm.
- Add the ability to detect and find the negative weight cycle in the Bellman-Ford algorithm.

P3128r2

- Add Oliver Rosten as contributor.

P3128r3

- Add a note that we will be unable to support a freestanding graph library in this proposal because of the need for **stack** , **queue** and potential **bad_alloc** exception in many of the algorithms.
- Restore use of exceptions in the algorithms since a freestanding implementation cannot supported.
- Add depth-first search algorithm to the Traversal section.
- Add visitor events for the depth-first search algorithm.
- Update the breadth-first search and topological sort algorithms to be in line with changes made to the Dijkstra and Bellman-Ford algorithms, such as adding a visitor parameter and overloads for multiple sources.

3 Algorithm Introduction

Basic characteristics of algorithms are summarized in tables of the following form:

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops Yes	Throws? No
---	---	--	-------------------

The parts of the table have the following meaning:

- **Complexity** The complexity of the algorithm based on the number of vertices (V) and edges (E).
- **Directed?** Is the algorithm only for directed graphs, or can it also be used for undirected graphs that have complimentary edges, with different directions, between two vertices.
- **Multi-edge?** Does the algorithm act as expected if more than one edge with the same direction exists between the same two vertices?

- **Cycles?** Does the algorithm act as expected if a vertex (or edge) is part of a cycle?
- **Self-loops?** Does the algorithm act as expected if an edge exists with the same source and target?
- **Throws?** Will the algorithm throw at all? If so, look at the *Throws* section after the function prototypes for details.

We are unable to support freestanding implementations in this proposal. Many of the algorithms require a `stack` or `queue`, which are not available in a freestanding environment. Additionally, `stack` and `queue` require memory allocation which could throw a `bad_alloc` exception.

4 Naming Conventions

Table 2 shows the naming conventions used throughout the Graph Library documents.

Template Parameter	Type Alias	Variable Names	Description
G			Graph
	<code>graph_reference_t<G></code>	<code>g</code>	Graph reference
GV		<code>val</code>	Graph Value, value or reference
EL		<code>el</code>	Edge list
V	<code>vertex_t<G></code>		Vertex descriptor
	<code>vertex_reference_t<G></code>	<code>u, v</code>	Vertex descriptor reference. <code>u</code> is the source (or only) vertex. <code>v</code> is the target vertex.
VId	<code>vertex_id_t<G></code>	<code>uid, vid, seed</code>	Vertex id. <code>uid</code> is the source (or only) vertex id. <code>vid</code> is the target vertex id.
VV	<code>vertex_value_t<G></code>	<code>val</code>	Vertex Value, value or reference. This can be either the user-defined value on a vertex, or a value returned by a function object (e.g. <code>VVF</code>) that is related to the vertex.
VR	<code>vertex_range_t<G></code>	<code>ur, vr</code>	Vertex Range
VI	<code>vertex_iterator_t<G></code>	<code>ui, vi</code>	Vertex Iterator. <code>ui</code> is the source (or only) vertex iterator. <code>vi</code> is the target vertex iterator.
		<code>first, last</code>	<code>first</code> and <code>last</code> are the begin and end iterators of a vertex range.
VVF		<code>vvf</code>	Vertex Value Function: <code>vvf(u) → vertex value</code> , or <code>vvf(uid) → vertex value</code> , depending on requirements of the consuming algorithm or view.
VProj		<code>vproj</code>	Vertex info projection function: <code>vproj(u) → vertex_info<VId, VV></code> .
	<code>partition_id_t<G></code>	<code>pid</code>	Partition id.
		<code>P</code>	Number of partitions.
PVR	<code>partition_vertex_range_t<G></code>	<code>pur, pvr</code>	Partition vertex range.
E	<code>edge_t<G></code>		Edge descriptor
	<code>edge_reference_t<G></code>	<code>uv, vw</code>	Edge descriptor reference. <code>uv</code> is an edge from vertices <code>u</code> to <code>v</code> . <code>vw</code> is an edge from vertices <code>v</code> to <code>w</code> .
EV	<code>edge_value_t<G></code>	<code>val</code>	Edge Value, value or reference. This can be either the user-defined value on an edge, or a value returned by a function object (e.g. <code>EVF</code>) that is related to the edge.
ER	<code>vertex_edge_range_t<G></code>		Edge Range for edges of a vertex
EI	<code>vertex_edge_iterator_t<G></code>	<code>uvi, vwi</code>	Edge Iterator for an edge of a vertex. <code>uvi</code> is an iterator for an edge from vertices <code>u</code> to <code>v</code> . <code>vwi</code> is an iterator for an edge from vertices <code>v</code> to <code>w</code> .
EVF		<code>evf</code>	Edge Value Function: <code>evf(uv) → edge value</code> .
EProj		<code>eproj</code>	Edge info projection function: <code>eproj(uv) → edge_info<VId, Sourced, EV></code> .

Table 2: Naming Conventions for Types and Variables

5 Algorithm Selection

When determining the algorithms to propose we split them into different tiers. Tier 1 algorithms are included in this proposal. The algorithms selected are a result of balancing a few things:

- Include a rich enough set of algorithms for the library to be useful.
- Include algorithms with well-defined functionality and agreed-upon algorithmic description.
- Don't include so many that the proposal will get bogged down for years and years.

5.1 Tier 1 Algorithms

<i>Traversal</i>	<i>Communities</i>	<i>Maximal Independent Set</i>
— Breadth-First search	— Label propagation	— Maximal independent set
— Depth-First search		<i>Link Analysis</i>
— Topological sort	<i>Components</i>	— Jaccard coefficient
	— Articulation points	<i>Minimal Spanning Tree</i>
<i>Shortest Paths</i>	— Connected components	— Kruskal's MST
— Dijkstra's algorithm	— Biconnected components	— Prim's MST
— Bellman-Ford algorithm	— Kosaraju's Strongly CC	
<i>Clustering</i>	— Tarjan's Strongly CC	
— Triangle counting		

Traversal and Shortest Paths algorithms include single-source and multi-source versions with multiple targets.

5.2 Other Algorithms

Additional algorithms that were considered but not included in this proposal are shown in Table 3. Tier X algorithms are variations of shortest paths algorithms that complement the Single Source, Multiple Target algorithms in this proposal. It is assumed that future proposals will include them, though the exact mix for each proposal will depend on feedback received and our experience with the current proposal.

Tier 2	Tier 3	Tier X
All Pairs Shortest Paths Floyd-Warshall Johnson Centrality: Betweenness Centrality Coloring: Greedy Communities: Louvain Connectivity: Minimum Cuts Transitive Closure Flows: Edmonds Karp Flows: Push Relabel Flows: Boykov Kolmogorov Link Analysis: Adamic-Adar Index Pathfinding: A* Best-first search	Jones Plassman Cores: k-cores Cores: k-truss Subgraph Isomorphism	Single Source, Single Target: Shortest Paths Driver Single Source, Single Target: BFS Single Source, Single Target: Dijkstra Single Source, Single Target: Bellman-Ford Single Source, Single Target: Delta Stepping Multiple Source: Shortest Paths Driver Multiple Source: BFS Multiple Source: Dijkstra Multiple Source: Bellman-Ford Multiple Source: Delta Stepping Multiple Source, Single Target: Shortest Paths Driver Multiple Source, Single Target: BFS Multiple Source, Single Target: Dijkstra Multiple Source, Single Target: Bellman-Ford Multiple Source, Single Target: Delta Stepping

Table 3: Other Algorithms

6 Common Algorithm Definitions

Common concepts used by algorithms are in this section, extending those in the Graph Container Interface.

6.1 Edge Weight Concepts

Edge weights are intrinsic numeric type for the current proposal, but could be any type in the future.

```
// For exposition only
template <class G, class WF, class DistanceValue, class Compare, class Combine>
concept basic_edge_weight_function = // e.g. weight(uv)
    is_arithmetic_v<DistanceValue> &&
    strict_weak_order<Compare, DistanceValue, DistanceValue> &&
    assignable_from<add_lvalue_reference_t<DistanceValue>,
        invoke_result_t<Combine, DistanceValue, invoke_result_t<WF, edge_reference_t<G>>>>;

// For exposition only
template <class G, class WF, class DistanceValue>
concept edge_weight_function = // e.g. weight(uv)
    is_arithmetic_v<invoke_result_t<WF, edge_reference_t<G>>> &&
    basic_edge_weight_function<G,
        WF,
        DistanceValue,
        less<DistanceValue>,
        plus<DistanceValue>>;
```

6.2 Visitor Concepts and Classes

Visitors are optional member functions on a user-defined class that are called during the execution of an algorithm. Each algorithm has its own set of visitor events that it supports, and each event function must match the visitor concepts shown in this section.

The visitor events mimic those used in the Boost Graph Library.

6.2.1 Vertex Visitor Concepts

```
template <class G, class Visitor>
concept has_on_initialize_vertex = // For exposition only
    requires(Visitor& v, vertex_info<vertex_id_t<G>, vertex_reference_t<G>, void> vdesc) {
        { v.on_initialize_vertex(vdesc) };
    };

template <class G, class Visitor>
concept has_on_discover_vertex = // For exposition only
    requires(Visitor& v, vertex_info<vertex_id_t<G>, vertex_reference_t<G>, void> vdesc) {
        { v.on_discover_vertex(vdesc) };
    };

template <class G, class Visitor>
concept has_on_start_vertex = // For exposition only
    requires(Visitor& v, vertex_info<vertex_id_t<G>, vertex_reference_t<G>, void> vdesc) {
        { v.on_start_vertex(vdesc) };
    };

template <class G, class Visitor>
concept has_on_examine_vertex = // For exposition only
    requires(Visitor& v, vertex_info<vertex_id_t<G>, vertex_reference_t<G>, void> vdesc) {
        { v.on_examine_vertex(vdesc) };
    };

template <class G, class Visitor>
concept has_on_finish_vertex = // For exposition only
```



```
requires(Visitor& v, vertex_info<vertex_id_t<G>, vertex_reference_t<G>, void> vdesc) {
    { v.on_finish_vertex(vdesc) };
};
```

The vertex events are called under the following conditions.

- `on_initialize_vertex(vdesc)` is called once for each vertex before the algorithm is run.
- `on_discover_vertex(vdesc)` is called once for each source vertex passed to the algorithm.
- `on_examine_vertex(vdesc)` is called for a vertex before any of its outgoing edges are examined. It is possible that it will be called multiple times for the same vertex if paths are found to it from other vertices with a shorter distance.
- `on_start_vertex(vdesc)` is called for a vertex that is being examined.
- `on_finish_vertex(vdesc)` is called for vertex that is being examined, after all its outgoing edges have been examined.

6.2.2 Edge Visitor Concepts

```
template <class G, class Visitor>
concept has_on_examine_edge = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_examine_edge(edesc) };
    };

template <class G, class Visitor>
concept has_on_edge_relaxed = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_edge_relaxed(edesc) };
    };

template <class G, class Visitor>
concept has_on_edge_not_relaxed = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_edge_not_relaxed(edesc) };
    };

template <class G, class Visitor>
concept has_on_edge_minimized = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_edge_minimized(edesc) };
    };

template <class G, class Visitor>
concept has_on_edge_not_minimized = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_edge_not_minimized(edesc) };
    };

    template <class G, class Visitor>
concept has_on_tree_edge = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_tree_edge(edesc) };
    };

template <class G, class Visitor>
concept has_on_back_edge = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_back_edge(edesc) };
    };

template <class G, class Visitor>
concept has_on_forward_or_cross_edge = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
```

```

    { v.on_forward_or_cross_edge(edesc) };
};
template <class G, class Visitor>
concept has_on_finish_edge = // For exposition only
    requires(Visitor& v, edge_info<vertex_id_t<G>, true, edge_reference_t<G>, void> edesc) {
        { v.on_finish_edge(edesc) };
    };
};

```

The edge events are called under the following conditions.

- `on_examine_edge(edesc)` is called for edge of the source vertex that is being examined.
- `on_edge_relaxed(edesc)` is called when the distance to the target vertex of the edge is relaxed, or decreased.
- `on_edge_not_relaxed(edesc)` is called when the distance to the target vertex of the edge is not relaxed, or not decreased.
- `on_edge_minimized(edesc)` is called when the distance to the target vertex of the edge is minimized, or decreased to the minimum value.
- `on_edge_not_minimized(edesc)` is called when the distance to the target vertex of the edge is not minimized, or not decreased to the minimum value.
- `on_tree_edge(edesc)` is called when the edge is added to the tree.
- `on_back_edge(edesc)` is called when the edge is a back edge.
- `on_forward_or_cross_edge(edesc)` is called when the edge is a forward or cross edge.
- `on_finish_edge(edesc)` is called when the edge is finished being examined.

6.2.3 Visitor Classes

`empty_visitor` is used when no visitor is needed. It is a no-op struct that does nothing.

```
struct empty_visitor {};
```

7 Traversal

7.1 Breadth-First Search

Compute the breadth-first path from one or more `source` vertices to all reachable vertices in `graph`.

Complexity $\mathcal{O}((E + V) \log V)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops Yes	Throws? Yes
--	---	--	--------------------

7.1.1 Single Source Breadth-First Search

```

template <index_adjacency_list G, class Visitor = empty_visitor>
void breadth_first_search(
    G&& g, // graph
    vertex_id_t<G> source, // starting vertex_id
    Visitor&& visitor = empty_visitor())

```

7.1.2 Multi-Source Breadth-First Search

```
template <index_adjacency_list G, input_range Sources, class Visitor = empty_visitor>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
void breadth_first_search(
    G&& g, // graph
    const Sources& sources,
    Visitor&& visitor = empty_visitor())
```

Preconditions:

- $0 \leq \text{source} < \text{num_vertices}(\text{graph})$ for the single-source version.
- $0 \leq \text{source} < \text{num_vertices}(\text{graph})$, for each `source` in `sources`, for the multi-source version.

Effects:

- (1.1) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_initialize_vertex`, `on_discover_vertex`, `on_examine_vertex`, `on_finish_vertex`, `on_examine_edge`, `on_edge_relaxed`, and `on_edge_not_relaxed`.

Throws:

- (2.1) — `out_of_range` is thrown when `source` is not in the range $0 \leq \text{source} < \text{num_vertices}(g)$.

Complexity:

- (3.1) — $\mathcal{O}((|E| + |V|) \log |V|)$ based on using the binary heap in `std::priority_queue`.
- (3.2) — An implementation may choose to use a Fibonacci heap for a complexity of $\mathcal{O}(|E| + |V| \log |V|)$.

Remarks:

- (4.1) — `dijkstra_shortest_paths` provides extended functionality if `breadth_first_search` doesn't have enough capability.

7.2 Depth-First Search

Traverse the depth-first path from one or more `source` vertices to all reachable vertices in `graph`.

Complexity $\mathcal{O}(V + E)$	Directed? Yes Multi-edge? Yes	Cycles? No Self-loops Yes	Throws? No
---	--	--	-------------------

7.2.1 Single Source Depth-First Search

```
template <index_adjacency_list G, class Visitor = empty_visitor>
void depth_first_search(
    G&& g, // graph
    vertex_id_t<G> source, // starting vertex_id
    Visitor&& visitor = empty_visitor())
```

Preconditions:

- $0 \leq \text{source} < \text{num_vertices}(\text{graph})$.

Effects:

- (1.1) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_initialize_vertex`, `on_start_vertex`, `on_discover_vertex`, `on_finish_vertex`, `on_examine_edge`, `on_tree_edge`, `on_back_edge`, `on_forward_or_cross_edge`, and `on_finish_edge`.

2 *Throws:*

- (2.1) — `out_of_range` is thrown when `source` is not in the range $0 \leq \text{source} < \text{num_vertices}(g)$.

3 *Complexity:*

- (3.1) — $\mathcal{O}(|V| + |E|)$.

7.3 Topological Sort

A linear ordering of vertices such that for every directed edge (u,v) from vertex u to vertex v, u comes before v in the ordering.

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops Yes	Throws? No
---	---	--	-------------------

7.3.1 Initialization

```
template <class Predecessors>
constexpr void init_topological_sort(Predecessors& predecessors) {
    std::ranges::iota(predecessors, 0); // exposition only
}
```

Effects:

- Each `predecessors[i]` is initialized to `i`.

7.3.2 Single Source Topological Sort

```
template <index_adjacency_list G, class Predecessors>
convertible_to<vertex_id_t<G>, range_value_t<Predecessors>>
void topological_sort(const G& graph,
                     vertex_id_t<G> source,
                     Predecessors& predecessors);
```

7.3.3 Multi-Source Topological Sort

```
template <index_adjacency_list G, input_range Sources, class Predecessors>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
         convertible_to<vertex_id_t<G>, range_value_t<Predecessors>>
void topological_sort(const G& graph,
                     const Sources& sources,
                     Predecessors& predecessors);
```

1 *Preconditions:*

- (1.1) — $0 \leq \text{source} < \text{num_vertices}(\text{graph})$ for the single-source version.
- (1.2) — $0 \leq \text{source} < \text{num_vertices}(\text{graph})$, for each `source` in `sources`, for the multi-source version.
- (1.3) — `predecessors[i] = i` for $0 \leq i < \text{num_vertices}(g)$.

2 *Effects:*

- (2.1) — If vertex with index `i` is reachable from vertex `source`, then `predecessors[i]` will contain the predecessor vertex of vertex `i`. Otherwise `predecessors[i]` will contain `i`.

3 *Throws:*

- (3.1) — An `out_of_range` exception is thrown in the following cases:
- (3.1.1) — `size(predecessor) < size(vertices(g))`
- (3.1.2) — `source` is not in the range `0 <= source < num_vertices(graph)`.

4 *Complexity:*

- (4.1) — $\mathcal{O}((|E| + |V|) \log |V|)$ based on using the binary heap in `std::priority_queue`.
- (4.2) — An implementation may choose to use a Fibonacci heap for a complexity of $\mathcal{O}(|E| + |V| \log |V|)$.

5 *Remarks:*

- (5.1) — Duplicate sources do not affect the algorithm's complexity or correctness.

8 Shortest Paths

8.1 Initialization

```
template <class DistanceValue>
constexpr auto shortest_path_infinite_distance() {
    return numeric_limits<DistanceValue>::max(); // exposition only
}

template <class DistanceValue>
constexpr auto shortest_path_zero() { return DistanceValue(); } // exposition only

template <class Distances>
constexpr void init_shortest_paths(Distances& distances) {
    // exposition only
    ranges::fill(distances,
        shortest_path_infinite_distance<ranges::range_value_t<Distances>>());
}

template <class Distances, class Predecessors>
constexpr void init_shortest_paths(Distances& distances, Predecessors& predecessors) {
    // exposition only
    init_shortest_paths_distances(distances);
    ranges::iota(predecessors, 0);
}
```

1 *Effects:*

- (1.1) — `init_shortest_paths(distances)` sets all elements in `distance` to `shortest_path_infinite_distance()`
- (1.2) — `init_shortest_paths(distances, predecessors)` does the same as `shortest_path_infinite_distance(distances)` and sets `predecessors[i] = i` for `i < size(predecessors)`.

2 *Returns:*

- (2.1) — `shortest_path_infinite_distance()` returns the largest distance value, typically `numeric_limits<DistanceValue>::max()` for numeric types.
- (2.2) — `shortest_path_zero()` returns a value for for a zero-length path, typically 0 for numeric types.

8.2 Dijkstra Shortest Paths and Shortest Distances

Compute the shortest path and associated distance from vertex `source` to all reachable vertices in `graph` using non-negative weights.

Complexity $\mathcal{O}((E + V) \log V)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops Yes	Throws? Yes
--	---	--	--------------------

Note that complexity may be $\mathcal{O}(|E| + |V| \log |V|)$ for certain implementations that use a Fibonacci heap instead of a binary heap implemented with `std::priority_queue`.

8.2.1 Dijkstra Shortest Paths

8.2.1.1 Single-Source Shortest Paths

```
template <index_adjacency_list G,
          random_access_range Distances,
          random_access_range Predecessors,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires is_arithmetic_v<range_value_t<Distances>> &&
        sized_range<Distances> &&
        sized_range<Predecessors> &&
        convertible_to<vertex_id_t<G>, range_value_t<Predecessors>> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr void dijkstra_shortest_paths(
    G&& g,
    const vertex_id_t<G> source,
    Distances& distances,
    Predecessors& predecessor,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());
```

8.2.1.2 Multi-Source Shortest Paths

```
template <index_adjacency_list G,
          input_range Sources,
          random_access_range Distances,
          random_access_range Predecessors,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
        is_arithmetic_v<range_value_t<Distances>> &&
        sized_range<Distances> &&
        sized_range<Predecessors> &&
        convertible_to<vertex_id_t<G>, range_value_t<Predecessors>> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr void dijkstra_shortest_paths(
    G&& g,
    const Sources& sources,
    Distances& distances,
    Predecessors& predecessor,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());
```

1 *Mandates:*

- (1.1) — `0 <= source < num_vertices(graph)` for the single-source version.
- (1.2) — `0 <= source < num_vertices(graph)`, for each `source` in `sources`, for the multi-source version.
- (1.3) — The weight function `w` must return a non-negative value.

2 *Preconditions:*

- (2.1) — `distances[i] = shortest_path_infinite_distance()` for `0 <= i < num_vertices(g)`.
- (2.2) — `predecessors[i] = i` for `0 <= i < num_vertices(g)`.

3 *Effects:*

- (3.1) — If vertex with index `i` is reachable from vertex `source`, then `distances[i]` will contain the distance from `source` to vertex `i`. Otherwise `distances[i]` will contain `shortest_path_infinite_distance()`.
- (3.2) — If vertex with index `i` is reachable from vertex `source`, then `predecessors[i]` will contain the predecessor vertex of vertex `i`. Otherwise `predecessors[i]` will contain `i`.
- (3.3) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_initialize_vertex`, `on_discover_vertex`, `on_examine_vertex`, `on_finish_vertex`, `on_examine_edge`, `on_edge_relaxed`, and `on_edge_not_relaxed`.

4 *Throws:*

- (4.1) — An `out_of_range` exception is thrown in the following cases:
 - (4.1.1) — `size(distances) < size(vertices(g))`
 - (4.1.2) — `size(predecessor) < size(vertices(g))`
 - (4.1.3) — `source` is not in the range `0 <= source < num_vertices(graph)`.
 - (4.1.4) — The weight function returns a negative value. This check is not made if the weight value type is an unsigned integral type.

5 *Complexity:*

- (5.1) — $\mathcal{O}((|E| + |V|) \log |V|)$ based on using the binary heap in `std::priority_queue`.
- (5.2) — An implementation may choose to use a Fibonacci heap for a complexity of $\mathcal{O}(|E| + |V| \log |V|)$.

6 *Remarks:*

- (6.1) — Duplicate sources do not affect the algorithm's complexity or correctness.
- (6.2) — Bellman-Ford Shortest Paths allows negative weights with the consequence of greater complexity.

8.2.2 Dijkstra Shortest Distances

This is the same as *Shortest Paths* except that it excludes the predecessors, giving a small performance improvement with lower memory overhead.

8.2.2.1 Single-Source Shortest Distances

```
template <index_adjacency_list G,
          random_access_range Distances,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires is_arithmetic_v<range_value_t<Distances>> &&
```

```

        sized_range<Distances> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr void dijkstra_shortest_distances(
    G&& g,
    const vertex_id_t<G> source,
    Distances& distances,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());

```

8.2.2.2 Multi-Source Shortest Distances

```

template <index_adjacency_list G,
        input_range Sources,
        random_access_range Distances,
        class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
        class Visitor = empty_visitor,
        class Compare = less<range_value_t<Distances>>,
        class Combine = plus<range_value_t<Distances>>>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
        sized_range<Distances> &&
        is_arithmetic_v<range_value_t<Distances>> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr void dijkstra_shortest_distances(
    G&&& g,
    const Sources& sources,
    Distances& distances,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());

```

1 *Mandates:*

- (1.1) — `0 <= source < num_vertices(graph)` for the single-source version.
- (1.2) — `0 <= source < num_vertices(graph)`, for each `source` in `sources`, for the multi-source version.
- (1.3) — The weight function `w` must return a non-negative value.

2 *Preconditions:*

- (2.1) — `distances[i] = shortest_path_infinite_distance()` for `0 <= i < num_vertices(g)`.

3 *Effects:*

- (3.1) — If vertex with index `i` is reachable from vertex `source`, then `distances[i]` will contain the distance from `source` to vertex `i`. Otherwise `distances[i]` will contain `shortest_path_infinite_distance()`.
- (3.2) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_initialize_vertex`, `on_discover_vertex`, `on_examine_vertex`, `on_finish_vertex`, `on_examine_edge`, `on_edge_relaxed`, and `on_edge_not_relaxed`.

4 *Throws:*

- (4.1) — An `out_of_range` exception is thrown in the following cases:
 - (4.1.1) — `size(distances) < size(vertices(g))`
 - (4.1.2) — `source` is not in the range `0 <= source < num_vertices(graph)`.

- (4.1.3) — The weight function returns a negative value. This check is not made if the weight value type is an unsigned integral type.

5 *Complexity:*

- (5.1) — $\mathcal{O}((|E| + |V|) \log |V|)$ based on using the binary heap in `std::priority_queue`.
- (5.2) — An implementation may choose to use a Fibonacci heap for a complexity of $\mathcal{O}(|E| + |V| \log |V|)$.

6 *Remarks:*

- (6.1) — Duplicate sources do not affect the algorithm's complexity or correctness.
- (6.2) — Bellman-Ford Shortest Distances allows negative weights with the consequence of greater complexity.

8.3 Bellman-Ford Shortest Paths and Shortest Distances

Compute the shortest path and associated distance from vertex `source` to all reachable vertices in `graph`.

Complexity $\mathcal{O}(E \cdot V)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops Yes	Throws? Yes
--	---------------------------------	------------------------------	-------------

The Bellman-Ford algorithm supports the use of negative edge weights, at cost in performance. Because of its complexity, it can only be used for small graphs. If a user can guarantee that a graph has positive edge weights then Dijkstra's algorithm provides far better performance.

There is a special case where edges form a negative cycle. "If a graph contains a 'negative cycle' (i.e. a cycle whose edges sum to a negative value) that is reachable from the source, then there is no cheapest path: any path that has a point on the negative cycle can be made cheaper by one more walk around the negative cycle. In such a case, the Bellman-Ford algorithm can detect and report the negative cycle." **Wikipedia** ([?])

`find_negative_cycle` can be called after calling `bellman_ford_shortest_paths` to get the vertex ids of the negative weight cycle.

8.3.1 Bellman-Ford Shortest Paths

8.3.1.1 Single-Source Shortest Paths

```
template <index_adjacency_list G,
          random_access_range Distances,
          random_access_range Predecessors,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires is_arithmetic_v<range_value_t<Distances>> &&
         convertible_to<vertex_id_t<G>, range_value_t<Predecessors>> &&
         sized_range<Distances> &&
         sized_range<Predecessors> &&
         basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr optional<vertex_id_t<G>> bellman_ford_shortest_paths(
    G&& g,
    const vertex_id_t<G> source,
    Distances& distances,
    Predecessors& predecessor,
    WF&& weight = [](edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());
```

8.3.1.2 Multi-Source Shortest Paths

```

template <index_adjacency_list G,
          input_range Sources,
          random_access_range Distances,
          random_access_range Predecessors,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
          is_arithmetic_v<range_value_t<Distances>> &&
          convertible_to<vertex_id_t<G>, range_value_t<Predecessors>> &&
          sized_range<Distances> &&
          sized_range<Predecessors> &&
          basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr optional<vertex_id_t<G>> bellman_ford_shortest_paths(
    G&& g,
    const Sources& sources,
    Distances& distances,
    Predecessors& predecessor,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());

```

Mandates:

- (1.1) — `0 <= source < num_vertices(graph)` for the single-source version.
- (1.2) — `0 <= source < num_vertices(graph)`, for each `source` in `sources`, for the multi-source version.

Preconditions:

- (2.1) — `distances[i] = shortest_path_infinite_distance()` for `0 <= i < num_vertices(g)`.
- (2.2) — `predecessors[i] = i` for `0 <= i < num_vertices(g)`.

Effects:

- (3.1) — If vertex with index `i` is reachable from vertex `source`, then `distances[i]` will contain the distance from `source` to vertex `i`. Otherwise `distances[i]` will contain `shortest_path_infinite_distance()`.
- (3.2) — If vertex with index `i` is reachable from vertex `source`, then `predecessors[i]` will contain the predecessor vertex of vertex `i`. Otherwise `predecessors[i]` will contain `i`.
- (3.3) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_examine_edge`, `on_edge_relaxed`, `on_edge_not_relaxed`, `on_edge_minimized`, and `on_edge_not_minimized`.

Returns:

- (4.1) — `optional<vertex_id_t<G>>` If no negative weight cycle is found, there is no associated vertex id. If a negative weight cycle is found, a vertex id in the cycle is returned. `find_negative_cycle` can be called to get the vertex ids of the cycle.

Throws:

- (5.1) — An `out_of_range` exception is thrown in the following cases:
 - (5.1.1) — `size(distances) < size(vertices(g))`
 - (5.1.2) — `source` is not in the range `0 <= source < num_vertices(graph)`.

6 *Complexity:* $\mathcal{O}(|E| \cdot |V|)$. Complexity may also be affected when visitor events are called.

7 *Remarks:*

(7.1) — Duplicate sources do not affect the algorithm's complexity or correctness.

(7.2) — Unlike Dijkstra's algorithm, Bellman-Ford allows negative edge weights. Performance constraints limit this to smaller graphs.

8.3.2 Bellman-Ford Shortest Distances

This is the same as *Shortest Paths* except that it excludes the predecessors, giving a small performance improvement with lower memory overhead.

8.3.2.1 Single-Source Shortest Distances

```
template <index_adjacency_list G,
          random_access_range Distances,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires is_arithmetic_v<range_value_t<Distances>> &&
        sized_range<Distances> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
constexpr optional<vertex_id_t<G>> bellman_ford_shortest_distances(
    G&& g,
    const vertex_id_t<G> source,
    Distances& distances,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());
```

8.3.2.2 Multi-Source Shortest Distances

```
template <index_adjacency_list G,
          input_range Sources,
          random_access_range Distances,
          class WF = function<range_value_t<Distances>(edge_reference_t<G>)>,
          class Visitor = empty_visitor,
          class Compare = less<range_value_t<Distances>>,
          class Combine = plus<range_value_t<Distances>>>
requires convertible_to<range_value_t<Sources>, vertex_id_t<G>> &&
        is_arithmetic_v<range_value_t<Distances>> &&
        sized_range<Distances> &&
        basic_edge_weight_function<G, WF, range_value_t<Distances>, Compare, Combine>
[[nodiscard]] constexpr optional<vertex_id_t<G>> bellman_ford_shortest_distances(
    G&& g,
    const Sources& sources,
    Distances& distances,
    WF&& weight = [] (edge_reference_t<G> uv) { return range_value_t<Distances>(1); },
    Visitor&& visitor = empty_visitor(),
    Compare&& compare = less<range_value_t<Distances>>(),
    Combine&& combine = plus<range_value_t<Distances>>());
```

1 *Mandates:*

(1.1) — $0 \leq \text{source} < \text{num_vertices}(\text{graph})$ for the single-source version.

(1.2) — $0 \leq \text{source} < \text{num_vertices}(\text{graph})$, for each `source` in `sources`, for the multi-source version.

2 *Preconditions:*

- (2.1) — `distances[i] = shortest_path_infinite_distance()` for $0 \leq i < \text{num_vertices}(g)$.

3 *Effects:*

- (3.1) — If vertex with index `i` is reachable from vertex `source`, then `distances[i]` will contain the distance from `source` to vertex `i`. Otherwise `distances[i]` will contain `shortest_path_infinite_distance()`.
- (3.2) — Member functions on the `visitor` parameter are called during the algorithm's execution. The functions are optional and, when included, must follow the visitor concepts for the events. No overhead is incurred if the functions are not included. The events supported are `on_examine_edge`, `on_edge_relaxed`, `on_edge_not_relaxed`, `on_edge_minimized`, and `on_edge_not_minimized`.

4 *Returns:*

- (4.1) — `optional<vertex_id_t<G>>` If no negative weight cycle is found, there is no associated vertex id. If a negative weight cycle is found, a vertex id in the cycle is returned. `bellman_ford_shortest_paths` must be used to get the predecessors if it is important to get the vertex ids of the cycle using `find_negative_cycle`.

5 *Throws:*

- (5.1) — An `out_of_range` exception is thrown in the following cases:
- (5.1.1) — `size(distances) < size(vertices(g))`
- (5.1.2) — `source` is not in the range $0 \leq \text{source} < \text{num_vertices}(\text{graph})$.

6 *Complexity:* $\mathcal{O}(|E| \cdot |V|)$. Complexity may also be affected when visitor events are called.

7 *Remarks:*

- (7.1) — Duplicate sources do not affect the algorithm's complexity or correctness.
- (7.2) — Unlike Dijkstra's algorithm, Bellman-Ford allows negative edge weights. Performance constraints limit this to smaller graphs.

8.3.3 Finding the Negative Cycle

If a cycle with negative weights is found, it's possible to get the vertex ids of the cycle using `find_negative_cycle` after calling `bellman_ford_shortest_paths`. It is not possible to get the cycle from `bellman_ford_shortest_distances` because it does not evaluate predecessors.

```
template <index_adjacency_list G, forward_range Predecessors, class OutputIterator>
requires output_iterator<OutputIterator, vertex_id_t<G>>
void find_negative_cycle(const G& g,
                       const Predecessors& predecessor,
                       const optional<vertex_id_t<G>>& cycle_vertex_id,
                       OutputIterator out_cycle);
```

1 *Preconditions:*

- (1.1) — `predecessors` must be evaluated by `bellman_ford_shortest_paths`.
- (1.2) — `cycle_vertex_id` is the return value of `bellman_ford_shortest_paths`.

2 *Effects:*

- (2.1) — All vertex ids in the negative weight cycle are written to the `out_cycle` output iterator.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

9 Clustering

9.1 Triangle Counting

Compute the number of triangles in a graph.

Complexity $\mathcal{O}(N^3)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops No	Throws? No
---	---	---	-------------------

```
template <index_adjacency_list G>
size_t triangle_count(G&& g);
```

- 1 *Preconditions:*
- (1.1) — The outgoing edges of a vertex are ordered by target_id.
- 2 *Returns:*
- (2.1) — Number of triangles
- 3 *Throws:*
- (3.1) — A `graph_error` is thrown when the target_id for an outgoing edge is less than the target_id of the previous edge.
- 4 *Complexity:* $\mathcal{O}(N^3)$
- 5 *Remarks:*
- (5.1) — To avoid duplicate counting, only directed triangles of a certain orientation will be detected. If `vertex_id(u) < vertex_id(v) < vertex_id(w)`, count triangle if graph contains edges `uv`, `vw`, `uw`.

10 Communities

10.1 Label Propagation

Propagate vertex labels by setting each vertex's label to the most popular label of its neighboring vertices. Every vertex voting on its new label represents one iteration of label propagation. Vertex voting order is randomized every iteration. The algorithm will iterate until label convergence, or optionally for a user specified number of iterations. Convergence occurs when no vertex label changes from the previous iteration. $\mathcal{O}(M)$ complexity is based on the complexity of one iteration, with number of iterations required for convergence considered small relative to graph size.

Some label propagation implementations use vertex ids as an initial labeling. This is not supported here because the label type can be more generic than the vertex id type. User is responsible for meaningful initial labeling.

Complexity $\mathcal{O}(M)$	Directed? Yes Multi-edge? Yes	Cycles? Yes Self-loops Yes	Throws? No
---------------------------------------	--	---	-------------------

```
template <index_adjacency_list G,
         ranges::random_access_range Label,
         class Gen = default_random_engine,
         class T = size_t>
void label_propagation(G&& g,
                      Label& label,
                      Gen&& rng = default_random_engine {},
                      T max_iters = numeric_limits<T>::max());
```

- 1 *Preconditions:*
- (1.1) — `label` contains initial vertex labels.

- (1.2) — `rng` is a random number generator for vertex voting order.
- (1.3) — `max_iters` is the maximum number of iterations of the label propagation, or equivalently the maximum distance a label will propagate from its starting vertex.

2 *Effects:* `label[uid]` is the label assignments of vertex id `uid` discovered by label propagation.

3 *Complexity:* $\mathcal{O}(M)$

4 *Remarks:* User is responsible for initial vertex labels.

Complexity $\mathcal{O}(M)$	Directed? Yes Multi-edge? Yes	Cycles? Yes Self-loops Yes	Throws? No
---------------------------------------	--	---	-------------------

```
template <index_adjacency_list G,
          ranges::random_access_range Label,
          class Gen = default_random_engine
          class T = size_t>
void label_propagation(G&& g,
                      Label& label,
                      ranges::range_value_t<Label>& empty_label,
                      Gen&& rng = default_random_engine {},
                      T max_iters = numeric_limits<T>::max());
```

5 *Preconditions:*

- (5.1) — `label` contains initial vertex labels.
- (5.2) — `empty_label` defines a label that is considered empty and will not be propagated.
- (5.3) — `rng` is a random number generator for vertex voting order.
- (5.4) — `max_iters` is the maximum number of iterations of the label propagation, or equivalently the maximum distance a label will propagate from its starting vertex.

6 *Effects:* `label[uid]` is the label assignments of vertex id `uid` discovered by label propagation.

7 *Complexity:* $\mathcal{O}(M)$

8 *Remarks:* User is responsible for initial vertex labels.

11 Components

11.1 Articulation Points

Find articulation points, or cut vertices, which when removed disconnect the graph into multiple components. Time complexity based on Hopcroft-Tarjan algorithm.

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? Yes Self-loops Yes	Throws? No
---	---	---	-------------------

```
template <index_adjacency_list G, class Iter>
requires output_iterator<Iter, vertex_id_t<G>>
void articulation_points(G&& g, Iter cut_vertices);
```

1 *Preconditions:*

- (1.1) — Output iterator `cut_vertices` can be assigned vertices of type `vertex_id_t<G>` when dereferenced.

2 *Effects:*

- (2.1) — Output iterator `cut_vertices` contains articulation point vertices, those which removed increase the number of components of `g`.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

11.2 BiConnected Components

Find the biconnected components, or maximal biconnected subgraphs of a graph, which are components that will remain connected if a vertex is removed. Time complexity based on Hopcroft-Tarjan algorithm.

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? Yes Self-loops Yes	Throws? No
---	---	---	-------------------

```
template <index_adjacency_list G,
          ranges::forward_range OuterContainer>
requires ranges::forward_range<ranges::range_value_t<OuterContainer>> &&
          integral<ranges::forward_range_t<ranges::forward_range_t<OuterContainer>>>
void biconnected_components(G&& g,
                           OuterContainer& components);
```

1 *Preconditions:*

(1.1) — `components` is a container of containers. The inner container stores vertex ids.

2 *Effects:*

(2.1) — `components` contains groups of biconnected components.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

11.3 Connected Components

Find weakly connected components of a graph. Weakly connected components are subgraphs where a path exists between all pairs of vertices when ignoring edge direction.

Complexity $\mathcal{O}(E + V)$	Directed? No Multi-edge? No	Cycles? Yes Self-loops Yes	Throws? No
---	--	---	-------------------

```
template <index_adjacency_list G,
          ranges::random_access_range Component>
void connected_components(G&& g,
                         Component& component);
```

1 *Preconditions:*

(1.1) — `size(component) >= num_vertices(g)`.

2 *Effects:*

(2.1) — `component[v]` is the connected component id of vertex `v`.

(2.2) — There is at least one Connected Component, with component id of 0, for `num_vertices(g) > 0`.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

11.4 Strongly Connected Components

11.4.1 Kosaraju's SCC

Find strongly connected components of a graph using Kosaraju's algorithm. Strongly connected components are subgraphs where a path exists between all pairs of vertices.

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? Yes Self-loops Yes	Throws? No
---	---	---	-------------------

```
template <index_adjacency_list G,
          index_adjacency_list GT,
          ranges::random_access_range Component>
void strongly_connected_components(G&& g,
                                  GT&& g_t,
                                  Component& component);
```

1 *Preconditions:*

- (1.1) — `g_t` is the transpose of `g`. Edge `uv` in `g` implies edge `vu` in `g_t`. `num_vertices(g)` equals `num_vertices(g_t)`.
- (1.2) — `size(component) >= num_vertices(g)`.

2 *Effects:*

- (2.1) — `component[v]` is the strongly connected component id of vertex `v`.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

11.4.2 Tarjan's SCC

Find strongly connected components of a graph using Tarjan's algorithm. Strongly connected components are subgraphs where a path exists between all pairs of vertices.

Complexity $\mathcal{O}(E + V)$	Directed? Yes Multi-edge? No	Cycles? Yes Self-loops Yes	Throws? No
---	---	---	-------------------

```
template <adjacency_list G,
          ranges::random_access_range Component>
requires ranges::random_access_range<vertex_range_t<G>> && integral<vertex_id_t<G>>
void strongly_connected_components(G&& g,
                                  Component& component);
```

1 *Preconditions:*

- (1.1) — `size(component) >= num_vertices(g)`.

2 *Effects:*

- (2.1) — `component[v]` is the strongly connected component id of `v`.

3 *Complexity:* $\mathcal{O}(|E| + |V|)$

12 Maximal Independent Set

12.1 Maximal Independent Set

Find a maximally independent set of vertices in a graph starting from a seed vertex. An independent vertex set indicates no pair of vertices in the set are adjacent.

```
template <index_adjacency_list G, class Iter>
requires output_iterator<Iter, vertex_id_t<G>>
void maximal_independent_set(G&& g, Iter mis, vertex_id_t<G> seed);
```


Complexity $\mathcal{O}(E)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops No	Throws? No
---	---	---	-------------------

1 *Preconditions:*

(1.1) — `0 <= seed < num_vertices(graph).`

(1.2) — `mis` output iterator can be assigned vertices of type `vertex_id_t<G>` when dereferenced.

2 *Effects:*

(2.1) — Output iterator `mis` contains maximal independent set of vertices containing `seed`, which is a subset of `vertices(graph).`

3 *Complexity:* $\mathcal{O}(|E|)$

13 Link Analysis

13.1 Jaccard Coefficient

Calculate the Jaccard coefficient of a graph

Complexity $\mathcal{O}(N ^3)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops No	Throws? No
---	---	---	-------------------

```
template <index_adjacency_list G, typename OutOp, typename T = double>
requires is_invocable_v<OutOp, vertex_id_t<G>&, vertex_id_t<G>&, edge_reference_t<G>, T>
void jaccard_coefficient(G&& g, OutOp out);
```

1 *Preconditions:*

(1.1) — `out` is an operator for setting the resulting Jaccard coefficient. This function is expected to be of the form `out(vertex_id_t<G> uid, vertex_id_t<G> vid, edge_t<G> uv, T val).`

2 *Effects:*

(2.1) — For every pair of neighboring vertices (`uid`, `vid`), the function `out` is called, passing the vertex ids, the edge `uv` between them, and the calculated Jaccard coefficient.

3 *Complexity:* $\mathcal{O}(|N|^3)$

14 Minimum Spanning Tree

14.1 Kruskal Minimum Spanning Tree

Find the minimum weight spanning tree of a graph using Kruskal's algorithm.

Complexity $\mathcal{O}(E)$	Directed? Yes Multi-edge? No	Cycles? No Self-loops No	Throws? No
---	---	---	-------------------

```
template <index_edgelist_range IELR, index_edgelist_range OELR>
void kruskal(IELR&& e, OELR&& t);

template <index_edgelist_range IELR, index_edgelist_range OELR T, CompareOp>
void kruskal(IELR&& e, OELR&& t, CompareOp compare);
```

- 1 *Preconditions:*
- (1.1) — `e` is an `edgelist`.
- (1.2) — `compare` operator is a valid comparison operation on two edge values of type `range_value_t<EL>::value_type` which returns a bool.
- 2 *Effects:*
- (2.1) — Edgelist `t` contains edges representing a spanning tree or forest, which minimize the comparison operator. When `compare` is `<`, `t` represents a minimum weight spanning tree.
- 3 *Complexity:* $\mathcal{O}(|E|)$

14.2 Prim Minimum Spanning Tree

Find the minimum weight spanning tree of a graph using Prim's algorithm.

Complexity $\mathcal{O}(E \log V)$	Directed? No Multi-edge? No	Cycles? No Self-loops No	Throws? No
--	--	---	-------------------

```
template <index_adjacency_list G,
          ranges::random_access_range Predecessor,
          ranges::random_access_range Weight>
void prim(G&& g, Predecessor& predecessor, Weight& weight, vertex_id_t<G> seed = 0);

template <index_adjacency_list G,
          ranges::random_access_range Predecessor,
          ranges::random_access_range Weight,
          class CompareOp>
void prim(G&& g,
          Predecessor& predecessor,
          Weight& weight,
          CompareOp compare,
          ranges::range_value_t<Weight> init_dist,
          vertex_id_t<G> seed = 0);
```

- 1 *Preconditions:*
- (1.1) — $0 \leq \text{seed} < \text{num_vertices}(g)$.
- (1.2) — Size of `weight` and `predecessor` is greater than or equal to `num_vertices(g)`.
- (1.3) — `compare` operator is a valid comparison operation on two edge values of type `edge_value_t<G>` which returns a bool.
- 2 *Effects:*
- (2.1) — `predecessor[v]` is the parent vertex of `v` in a tree rooted at `seed` and `weight[v]` is the value of the edge between `v` and `predecessor[v]` in the tree. When `compare` is `<` and `init_dist==+inf`, `predecessor` represents a minimum weight spanning tree.
- (2.2) — If `predecessor` and `weight` are not initialized by user, and the graph is not fully connected, `predecessor[v]` and `weight[v]` will be undefined for vertices not in the same connected component as `seed`.
- 3 *Complexity:* $\mathcal{O}(|E|\log|V|)$

Acknowledgements

Phil Ratzloff's time was made possible by SAS Institute.

Portions of *Andrew Lumsdaine's* time was supported by NSF Award OAC-1716828 and by the Segmented Global Address Space (SGAS) LDRD under the Data Model Convergence (DMC) initiative at the U.S. Department of Energy's Pacific Northwest National Laboratory (PNNL). PNNL is operated by Battelle Memorial Institute under Contract DE-AC06-76RL01830.

Michael Wong's work is made possible by Codeplay Software Ltd., ISOCPP Foundation, Khronos and the Standards Council of Canada.

Muhammad Osama's time was made possible by Advanced Micro Devices, Inc.

The authors thank the members of SG19 and SG14 study groups for their invaluable input.